



Breezy

Réseau social léger & scalable — Clone Twitter/X

Livrable : Rapport de projet

Conception et réalisation d'une application web distribuée

Juin 2026

Auteurs :

Marine MAZOU
Stéphane PLATHEY-BADIN
Nolan JOURDAN

Équipe : Groupe 4 — DAD

Formation : Ingénieur en informatique

École : CESI

Année : 2025 — 2026

TABLE OF CONTENTS

Introduction	3
1 Objectifs et attentes du projet	4
1.1 Contexte et problématique	4
1.2 Objectifs clés	4
1.3 Périmètre fonctionnel attendu	4
1.4 Livrables attendus	4
2 Architecture du système	5
2.1 Vue d'ensemble	5
2.2 Justification des choix d'architecture	6
2.3 Modèle de données	6
2.3.1 Dictionnaire des données — PostgreSQL	8
2.3.2 Dictionnaire des données — MongoDB	9
2.3.3 Associations et cardinalités (notation Merise)	11
2.3.4 Règles de validation applicatives	11
3 Hiérarchisation des tâches et fonctionnalités	11
3.1 Matrice des rôles et permissions	12
3.2 Fonctionnalités additionnelles	12
3.3 Répartition des tâches dans l'équipe	13
4 Étapes et ressources mobilisées	14
4.1 Phases du projet	14
4.2 Ressources mobilisées	14
5 Méthodologie	15
5.1 Démarche itérative	15
5.2 Organisation Git	15
5.3 Intégration et déploiement continu	15
6 Interfaces utilisateur — maquettes	16
7 Fonctionnalités principales et mise en œuvre	17
7.1 Authentification et gestion des rôles	17
7.2 Publication et fil d'actualité	17
7.3 Interactions sociales	17
7.4 Blocage et confidentialité	18
7.5 Notifications et messagerie	18
7.6 Modération	18
7.7 Couche d'intégration front/back	19
8 Axes d'amélioration et évolutions	20
Conclusion	21

INTRODUCTION

Ce rapport présente la conception et la réalisation de **Breezy**, un réseau social léger inspiré de Twitter/X, développé dans le cadre du projet de fin de module en ingénierie informatique au CESI. Le document détaille les objectifs poursuivis, l'architecture technique retenue, la méthodologie de travail mise en œuvre par l'équipe, ainsi que les fonctionnalités livrées et leurs perspectives d'évolution.

Le projet a été mené par une équipe de trois personnes aux rôles complémentaires : développement back-end, développement front-end, et intégration Full-Stack / DevOps. La répartition détaillée des responsabilités est présentée à la section 3.3.

1 OBJECTIFS ET ATTENTES DU PROJET

1.1 CONTEXTE ET PROBLÉMATIQUE

Les réseaux sociaux traditionnels sont souvent lourds et gourmands en ressources, ce qui dégrade l'expérience sur les terminaux modestes ou les connexions limitées. Breezy répond à ce constat en proposant un réseau social **léger, réactif et hautement scalable**, conçu selon une approche *mobile-first*, tout en reposant sur une architecture back-end robuste capable de monter en charge.

1.2 OBJECTIFS CLÉS

- **Performance et réactivité** sur l'ensemble des appareils, y compris les environnements à faibles ressources.
- **Architecture distribuée et scalable**, où chaque domaine fonctionnel est isolé et peut évoluer indépendamment.
- **Expérience utilisateur fluide**, pensée d'abord pour le mobile puis déclinée en interface responsive multi-format.
- **Sécurité renforcée**, fondée sur l'authentification par jeton JWT et une gestion stricte des rôles et permissions.

1.3 PÉRIMÈTRE FONCTIONNEL ATTENDU

Le cahier des charges définit **23 fonctionnalités** (notées Fx1 à Fx23), encadrées par une matrice de permissions associant chaque action à un niveau de rôle. Elles couvrent la gestion de compte, la publication de contenu (texte, images, vidéos), les interactions sociales (likes, commentaires, abonnements), la messagerie privée, les notifications, la modération et le confort d'usage (multi-langues, thème clair/sombre).

1.4 LIVRABLES ATTENDUS

- Une application web fonctionnelle, conteneurisée et déployable en une commande.
- Un code source versionné, organisé en monorepo et documenté.
- Le présent rapport de projet et le support de soutenance.

2 ARCHITECTURE DU SYSTÈME

2.1 VUE D'ENSEMBLE

L'application repose sur une **architecture distribuée en microservices**, orchestrée par Docker. Le client web (Next.js) communique exclusivement à travers une **passerelle API (Nginx)** qui assure le routage, la gestion du CORS, la limitation de débit et la distribution des médias statiques. Chaque microservice est responsable d'un domaine métier et persiste ses données dans la base la plus adaptée à sa nature.

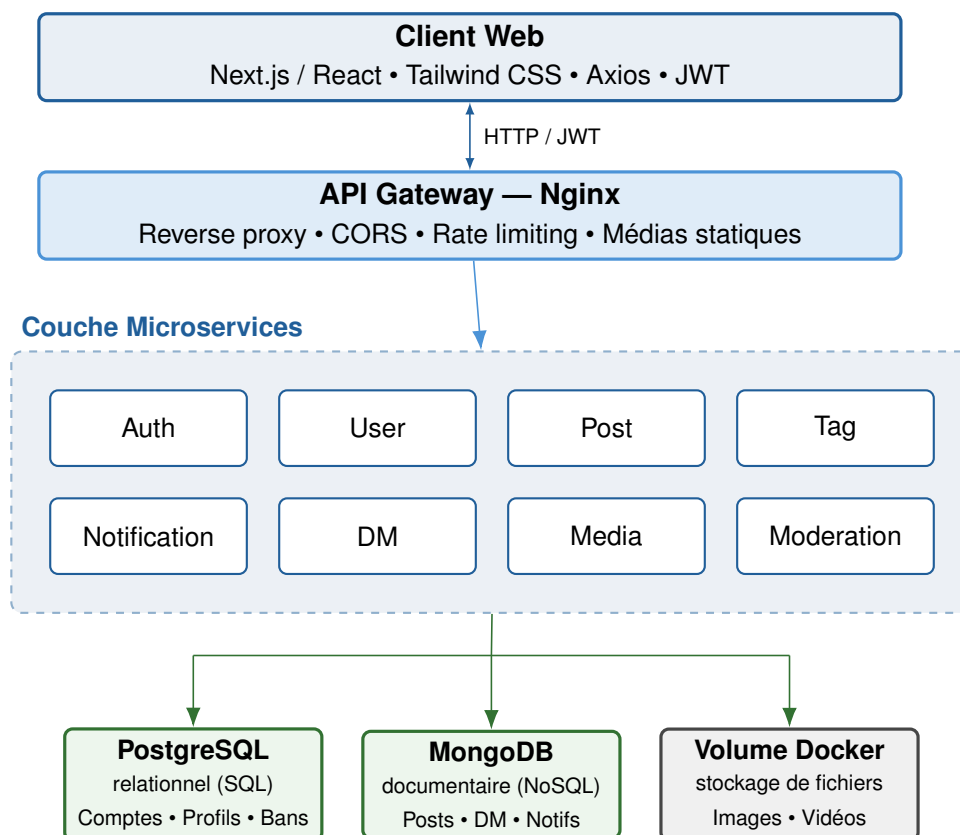


Figure 1: Architecture microservices de Breezy : client, passerelle Nginx, huit services métier et persistance polyglotte. **Deux bases de données** — PostgreSQL (relationnel) et MongoDB (documentaire) — accompagnées d'un **volume Docker** servant uniquement au stockage des fichiers médias (images, vidéos), qui n'est pas une base de données.

2.2 JUSTIFICATION DES CHOIX D'ARCHITECTURE

Microservices. Le découpage en huit services indépendants (Auth, User, Post, Tag, Notification, DM, Media, Moderation) procure l'isolation des pannes, le déploiement indépendant et une montée en charge ciblée sur les services les plus sollicités.

Passerelle API. Nginx centralise les préoccupations transverses (sécurité, CORS, limitation de débit, journalisation) et masque la topologie interne au client, qui ne connaît qu'un point d'entrée unique.

Persistance polyglotte. L'application s'appuie sur **deux bases de données**. Les données relationnelles fortement structurées (comptes, profils, abonnements, bannissements) sont confiées à **PostgreSQL** (SQL) via Sequelize, tandis que les données documentaires à fort volume et au schéma souple (posts, commentaires imbriqués, tags, messages, notifications, signalements) sont stockées dans **MongoDB** (NoSQL) via Mongoose.

Stockage des médias. Les **fichiers** images et vidéos ne sont pas placés en base de données : ils sont déposés par le `media-service` sur un **volume Docker** (système de fichiers persistant) et servis en statique. Seule l'**URL** du média est conservée en base, dans le champ `media` du document du post (MongoDB). Cette séparation garde les bases légères et délègue le service des fichiers volumineux au système de fichiers.

2.3 MODÈLE DE DONNÉES

Le modèle suit l'**architecture polyglotte** décrite plus haut : **PostgreSQL** porte l'identité et les relations sociales (Users, Profiles, Followers, BlockedUsers, Bans) ; **MongoDB** porte le contenu à fort volume ou imbriqué (posts, directmessages, notifications, reports). Les documents Mongo référencent l'identifiant `Users.id` via des champs `user_id`, `sender_id`, etc. : ce sont des **références logiques** (frontière inter-SGBD), dont l'intégrité est garantie au niveau applicatif et non par des clés étrangères.

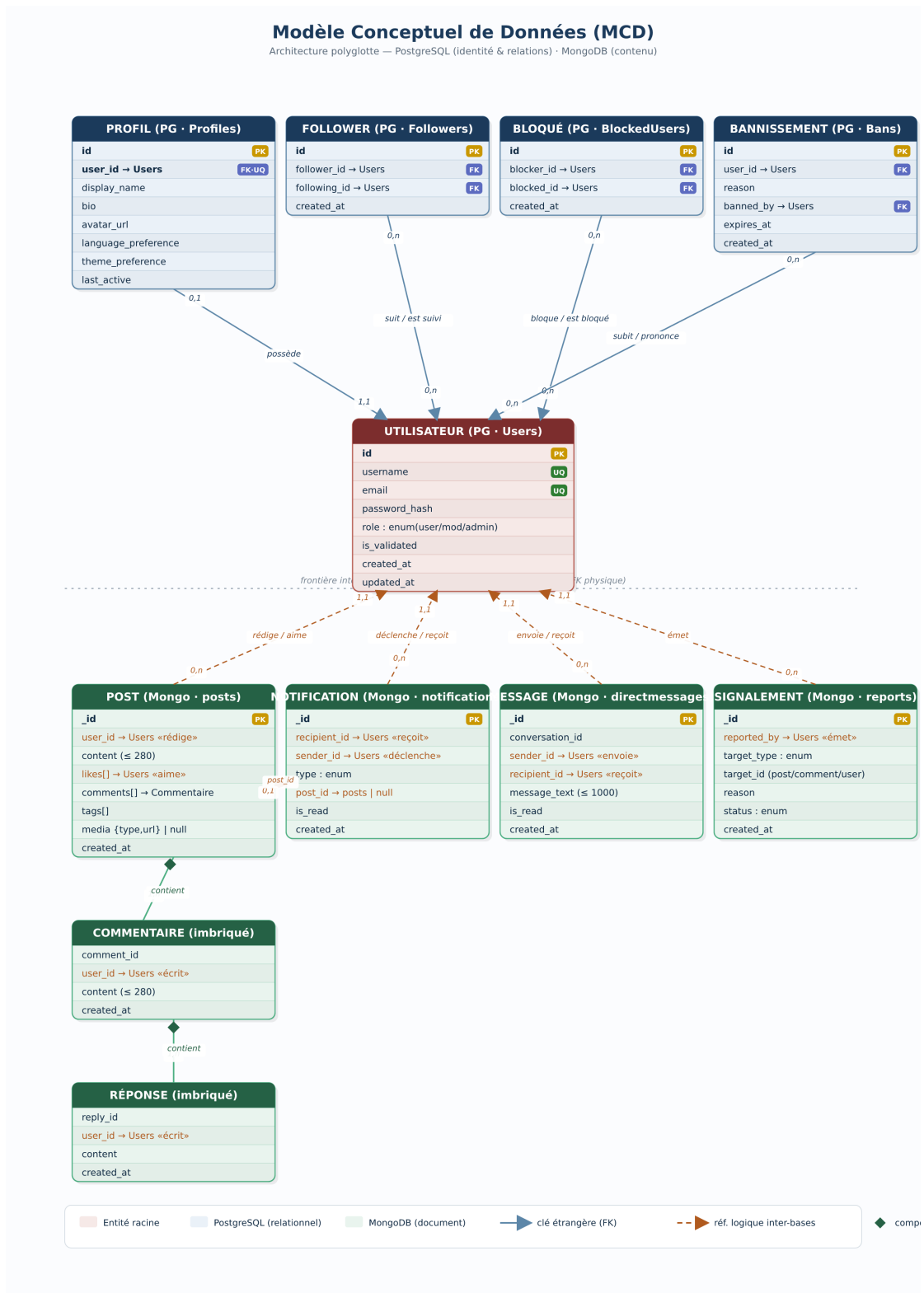


Figure 2: Modèle Conceptuel de Données (MCD) de Breezy. Entités PostgreSQL (violet) et MongoDB (vert) ; trait plein = clé étrangère réelle, trait orange pointillé = référence logique inter-bases (Users.id), losange = composition (sous-document imbriqué).

2.3.1 Dictionnaire des données — PostgreSQL

Table Users — comptes utilisateurs (entité racine de l'identité).

Champ	Type	Clé / Null	Description
id	INTEGER	PK, auto	Identifiant unique du compte, référencé par toutes les autres entités.
username	VARCHAR(50)	UQ, non null	Identifiant public (@pseudo), stocké en minuscules.
email	VARCHAR(255)	UQ, non null	Adresse e-mail de connexion.
password_hash	VARCHAR(255)	non null	Empreinte bcrypt du mot de passe (jamais stocké en clair).
role	ENUM (user / moderator / admin)	non null, déf. user	Rôle / niveau de privilège.
is_validated	BOOLEAN	non null, déf. true	Compte validé / actif.
created_at, updated_at	TIMESTAMP	non null	Dates de création et de dernière modification.

Table Profiles — informations de profil (relation 1–1 avec Users).

Champ	Type	Clé / Null	Description
id	INTEGER	PK, auto	Identifiant du profil.
user_id	INTEGER	FK→Users, UQ	Compte propriétaire (ON DELETE CASCADE). Unique ⇒ relation 1–1.
display_name	VARCHAR(100)	null	Nom d'affichage.
bio	TEXT	null	Biographie / description.
avatar_url	VARCHAR(500)	null	URL de la photo de profil.
language_preference	VARCHAR(10)	non null, déf. en	Langue de l'interface (fr / en / es).
theme_preference	VARCHAR(20)	non null, déf. light	Thème (light / dark).
last_active	TIMESTAMP	null	Dernière activité (présence en ligne).

Table Followers — abonnements (réflexive sur Users, unicité sur le couple).

Champ	Type	Clé / Null	Description
id	INTEGER	PK, auto	Identifiant de la relation.
follower_id	INTEGER	FK→Users	Utilisateur qui suit (CASCADE).
following_id	INTEGER	FK→Users	Utilisateur suivi (CASCADE).
created_at	TIMESTAMP	non null	Date de l'abonnement. Unicité (follower_id, following_id).

Table BlockedUsers — blocages (réflexive sur Users, unicité sur le couple).

Champ	Type	Clé / Null	Description
id	INTEGER	PK, auto	Identifiant du blocage.
blocker_id	INTEGER	FK→Users	Utilisateur qui bloque (CASCADE).
blocked_id	INTEGER	FK→Users	Utilisateur bloqué (CASCADE).
created_at	TIMESTAMP	non null	Date du blocage. Unicité (blocker_id, blocked_id).

Table Bans — bannissements (action de modération).

Champ	Type	Clé / Null	Description
id	INTEGER	PK, auto	Identifiant du bannissement.
user_id	INTEGER	FK→Users	Utilisateur banni (CASCADE).
reason	TEXT	non null	Motif du bannissement.
banned_by	INTEGER	FK→Users	Modérateur / admin auteur (CASCADE).
expires_at	TIMESTAMP	null	Fin du bannissement (NULL = permanent).
created_at	TIMESTAMP	non null	Date du bannissement.

2.3.2 Dictionnaire des données — MongoDB

Toutes les collections possèdent une clé technique `_id` (ObjectId) générée automatiquement ; les champs `*_id` numériques sont des références logiques vers `Users.id`.

Collection posts — publications.

Champ	Type	Clé / Idx	Description
_id	ObjectId	PK	Identifiant du post.
user_id	Number	IX, →Users	Auteur de la publication.
content	String	—	Texte du post (max 280 caractères).
likes	[Number]	—	Liste des <code>Users.id</code> ayant aimé (N–N « aime »).
comments	[Comment]	—	Commentaires imbriqués (sous-documents).
tags	[String]	IX	Hashtags associés.
media	PostMedia null	—	Média joint (image / vidéo) ou null.
created_at	Date	IX	Date de publication.

Index composés : {user_id, created_at} (fil d'un utilisateur), {tags, created_at} (recherche par tag).

Collection directmessages — messages privés.

Champ	Type	Clé / Idx	Description
_id	ObjectId	PK	Identifiant du message.
conversation_id	String	IX	Identifiant déterministe de la conversation (paire ordonnée).
sender_id	Number	→Users	Expéditeur.
recipient_id	Number	→Users	Destinataire.
message_text	String	—	Contenu (max 1000 caractères).
is_read	Boolean	—	Message lu par le destinataire.
created_at	Date	—	Date d'envoi.

Collection notifications — notifications.

Champ	Type	Clé / Idx	Description
_id	ObjectId	PK	Identifiant de la notification.
recipient_id	Number	IX, →Users	Destinataire de la notification.
sender_id	Number	→Users	Utilisateur à l'origine de l'événement.
type	String (enum)	—	mention / like / follow / dm / comment.
post_id	ObjectId	ref posts	Post concerné, le cas échéant (null sinon).
is_read	Boolean	IX	Notification lue.
created_at	Date	—	Date de l'événement.

Collection reports — signalements.

Champ	Type	Clé / Idx	Description
_id	ObjectId	PK	Identifiant du signalement.
reported_by	Number	→Users	Utilisateur signalant.
target_type	String (enum)	—	Cible : post / comment / user.
target_id	String	IX	Identifiant de la cible (ObjectId ou Users.id en chaîne).
reason	String	—	Motif du signalement.
status	String (enum)	IX	pending / resolved.
created_at	Date	—	Date du signalement.

Sous-documents imbriqués — Comment (dans posts.comments) : comment_id, user_id, content (max 280), created_at, replies[] ; Reply (dans Comment.replies) : reply_id, user_id, content, created_at ; PostMedia (dans posts.media) : type (image/video), url.

2.3.3 Associations et cardinalités (notation Merise)

Association	Cardinalités	Entités / remarques
POSSEDE	(1,1) – (0,1)	UTILISATEUR – PROFIL
SUIT	(0,n) – (0,n)	UTILISATEUR – UTILISATEUR (réflexive, table Followers)
BLOQUE	(0,n) – (0,n)	UTILISATEUR – UTILISATEUR (réflexive, table BlockedUsers)
REDIGE	(0,n) – (1,1)	UTILISATEUR – POST
AIME	(0,n) – (0,n)	UTILISATEUR – POST (tableau likes)
CONTIENT	(1,1) – (0,n)	POST – COMMENTAIRE (composition, entité faible)
ECRIT_COM	(0,n) – (1,1)	UTILISATEUR – COMMENTAIRE
CONTIENT	(1,1) – (0,n)	COMMENTAIRE – REPONSE (composition, entité faible)
ECRIT_REP	(0,n) – (1,1)	UTILISATEUR – REPONSE
ENVOIE / RECOIT	(0,n) – (1,1)	UTILISATEUR – MESSAGE (deux rôles)
DECLENCHE / RECOIT	(0,n) – (1,1)	UTILISATEUR – NOTIFICATION (porte type, is_read)
CONCERNE	(0,1) – (0,n)	NOTIFICATION – POST (optionnel)
EMET_SIGN	(0,n) – (1,1)	UTILISATEUR – SIGNALEMENT
CIBLE	(1,1) – (0,n)	SIGNALEMENT – cible polymorphe (post / commentaire / user)
SUBIT / PRONONCE	(0,n) – (1,1)	UTILISATEUR – BANNISSEMENT (banni user_id, auteur banned_by)

Les associations **SUIT** et **BLOQUE** sont réflexives et porteuses (matérialisées par des tables d'association avec contrainte d'unicité). **CIBLE** est **polymorphe** : le couple (target_type, target_id) désigne un post, un commentaire ou un utilisateur, et n'est donc pas contraignable par une FK classique. **COMMENTAIRE** et **REPONSE** sont des entités faibles (documents imbriqués).

2.3.4 Règles de validation applicatives

En complément des contraintes du SGBD, la couche service applique des contrôles avant insertion :

Donnée	Règles
username	3–30 caractères, jeu [a-z0-9_.] ; ne commence/fini pas par . ou _ ; pas de .. ; pseudos réservés interdits.
email	Format local@domaine.tld, sans espace, ≤ 254 caractères.
password	8–128 caractères ; au moins une minuscule, une majuscule, un chiffre et un caractère spécial.
contenu post / commentaire	Obligatoire, non vide, ≤ 280 caractères.
message privé	Destinataire numérique requis ; texte non vide, ≤ 1000 caractères.
signalement	target_type, target_id et reason requis.
bannissement	user_id et reason requis.

3 HIÉRARCHISATION DES TÂCHES ET FONCTIONNALITÉS

3.1 MATRICE DES RÔLES ET PERMISSIONS

L'accès aux fonctionnalités est gouverné par une hiérarchie de quatre rôles : **Visiteur** < **Utilisateur** < **Modérateur** < **Administrateur**. Cette matrice constitue la colonne vertébrale fonctionnelle du projet et a guidé la priorisation du développement.

#	Fonctionnalité	Vis.	Util.	Mod.	Admin
Fx1	Création de compte avec validation	✓	—	—	✓
Fx2	Authentification sécurisée (JWT)	—	✓	✓	✓
Fx3	Publication de messages (280 car.)	—	✓	✓	✓
Fx4–5	Profil & flux chronologique	—	✓	✓	✓
Fx6–8	Liker, commenter, répondre	—	✓	✓	✓
Fx9–11	Suivre, profils, posts d'un user	—	✓	✓	✓
Fx12–13	Tags et recherche par tag	—	✓	✓	✓
Fx14–16	Notifications (mention/like/follow)	—	✓	✓	✓
Fx17	Messages privés	—	✓	✓	✓
Fx18–19	Images et vidéos dans les messages	—	✓	✓	✓
Fx20	Signalement de contenu	—	✓	✓	✓
Fx21	Suspension / bannissement	—	—	✓	✓
Fx22	Interface multi-langues	—	✓	✓	✓
Fx23	Thème sombre / clair	✓	✓	✓	✓

Table 1: Matrice des permissions par rôle (23 fonctionnalités).

3.2 FONCTIONNALITÉS ADDITIONNELLES

Au-delà des 23 fonctionnalités du cahier des charges, l'équipe a livré plusieurs fonctionnalités complémentaires renforçant l'expérience et la robustesse :

- **Blocage d'utilisateur** (désabonnement mutuel, messages et posts masqués entre comptes bloqués) ;
- **Refus de connexion des comptes bannis** (bannissement appliqué à l'authentification) ;
- **Suggestions** de comptes à suivre ;
- **Présence en ligne** et **accusés de lecture** dans la messagerie, **pastilles** de notifications et de messages non lus ;
- **Renouvellement silencieux** de session (refresh token) ;
- **Recherche en temps réel** (au fil de la frappe, par tag ou utilisateur) et **posts tendance**.

Les fonctionnalités ont été hiérarchisées selon la méthode MoSCoW afin de sécuriser un socle livrable avant d'enrichir le périmètre :

- **Must have** — authentification JWT, publication, feed, likes, commentaires, abonnements, profils (cœur du réseau social).

- **Should have** — tags et recherche, notifications, messages privés, médias images.
- **Could have** — vidéos, modération avancée, multi-langues, thème clair/sombre.
- **Won't have (pour cette itération)** — appels audio/vidéo, recommandation algorithmique du fil.

3.3 RÉPARTITION DES TÂCHES DANS L'ÉQUIPE

Membre	Rôle	Responsabilités principales
Stéphane PLATHEY-BADIN	Back-end	Microservices, modèles de données, API REST, tests unitaires et d'intégration, sécurité applicative.
Nolan JOURDAN	Front-end	Interfaces Next.js, composants, design system Tailwind, maquettes et ergonomie mobile-first.
Marine MAZOU	Full-Stack, DevOps & Chef de projet	Liaison front-end / back-end, conteneurisation Docker, passerelle Nginx, déploiement, coordination et suivi de l'avancement.

Table 2: Répartition des responsabilités au sein de l'équipe.

4 ÉTAPES ET RESSOURCES MOBILISÉES

4.1 PHASES DU PROJET

1. **Cadrage** — analyse du besoin, définition des 23 fonctionnalités et de la matrice de permissions, choix de la stack.
2. **Conception** — modélisation des données, découpage en microservices, définition des contrats d'API.
3. **Développement back-end** — implémentation des services, des modèles Sequelize/Mongoose et des routes REST.
4. **Développement front-end** — construction des écrans, du design system et du rendu responsive.
5. **Intégration** — liaison front/back via la couche de services Axios, gestion du JWT, harmonisation des contrats.
6. **Déploiement & recette** — conteneurisation, orchestration Docker Compose, jeu de données de démonstration, tests de bout en bout.

4.2 RESSOURCES MOBILISÉES

Catégorie	Détail
Humaines	Équipe de 3 développeurs aux rôles complémentaires.
Langages	TypeScript (back), JavaScript / JSX (front).
Frameworks	Express.js, Next.js / React, Tailwind CSS.
Données	PostgreSQL 15 + Sequelize, MongoDB 6 + Mongoose.
Infrastructure	Docker, Docker Compose, Nginx, npm Workspaces.
Sécurité	JWT (HS256) en cookies <code>httpOnly</code> (access + refresh), bcrypt, RBAC, en-têtes de sécurité Nginx, limitation de débit.
Outils	Git / GitHub, GitHub Actions (CI), Adminer, mongo-express.

Table 3: Ressources techniques et humaines mobilisées.

5 MÉTHODOLOGIE

5.1 DÉMARCHE ITÉRATIVE

Le projet a suivi une démarche **agile et itérative** : le périmètre a été livré par incréments fonctionnels, chaque itération ajoutant un ensemble cohérent de fonctionnalités testables. Cette approche a permis de disposer rapidement d'un socle démontrable et d'absorber les ajustements (vidéos, gestion fine des rôles) sans remettre en cause l'architecture.

5.2 ORGANISATION GIT

Le code est versionné dans un **monorepo** géré avec npm Workspaces. Le travail s'appuie sur un modèle de branches par fonctionnalité :

- `main` — version stable et déployable.
- `dev` — branche d'intégration consolidant le back-end et le front-end.
- `feat/*` — une branche par fonctionnalité ou chantier d'intégration.

Les messages de commit respectent la convention **Conventional Commits** (`feat`, `fix`, `chore`, `docs`...), ce qui rend l'historique lisible et facilite la relecture entre pairs via les *pull requests*.

5.3 INTÉGRATION ET DÉPLOIEMENT CONTINUS

Une chaîne **CI** (GitHub Actions) exécute la compilation du package partagé puis les tests unitaires et d'intégration de chaque service. Le déploiement local est entièrement automatisé : une seule commande `docker compose up` démarre les bases de données, les huit services, la passerelle Nginx et le front-end.

6 INTERFACES UTILISATEUR — MAQUETTES

L'interface a été conçue *mobile-first* puis déclinée en version responsive « trois colonnes » sur grand écran, à la manière de X. Les wireframes ci-dessous synthétisent les deux dispositions principales.

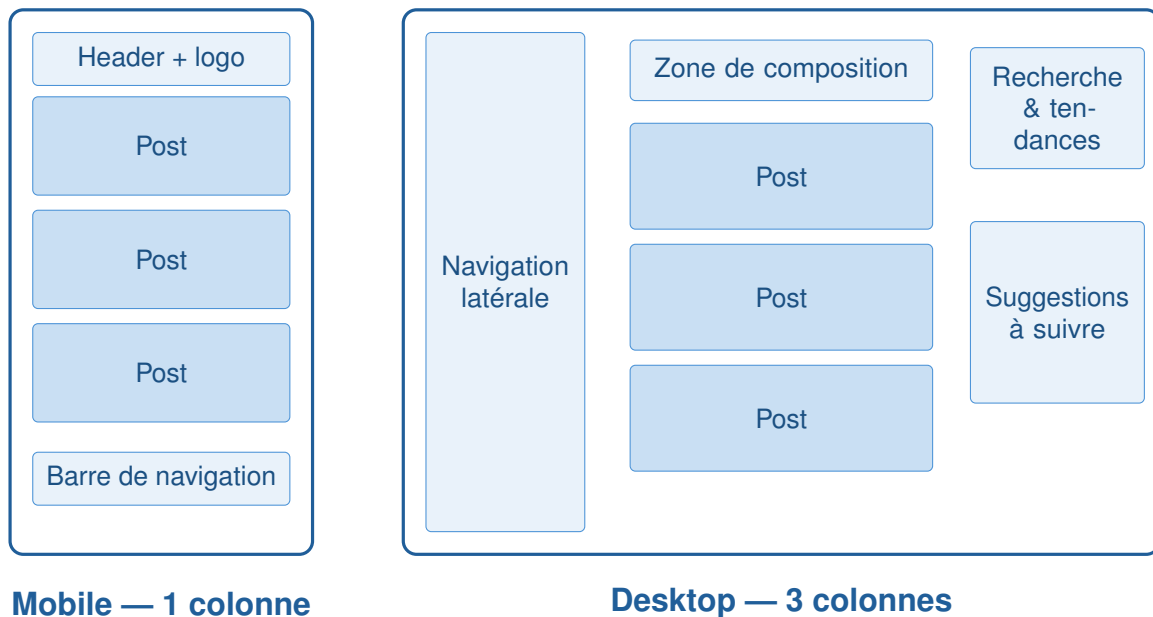


Figure 3: Wireframes des dispositions mobile (une colonne, navigation basse) et bureau (trois colonnes : navigation, fil central, tendances).

Principes d'ergonomie. La navigation principale passe d'une barre inférieure sur mobile à une barre latérale sur bureau ; le fil central reste l'élément focal ; la colonne de droite (recherche, tendances, suggestions) n'apparaît qu'à partir des grands écrans. Un thème clair/sombre et un sélecteur de langue (FR/EN/ES) sont accessibles depuis l'interface.

7 FONCTIONNALITÉS PRINCIPALES ET MISE EN ŒUVRE

7.1 AUTHENTIFICATION ET GESTION DES RÔLES

L'inscription crée un compte ; la connexion délivre deux jetons **JWT** (signés HS256) déposés en cookies `httpOnly` : un **jeton d'accès** de courte durée (1 h) et un **jeton de rafraîchissement** de 7 jours. Le choix d'une durée d'accès courte est volontaire : il **limite la fenêtre d'exploitation** en cas de vol du jeton, tout en préservant le confort grâce au rafraîchissement **silencieux** — à l'expiration du jeton d'accès, le client obtient automatiquement un nouveau jeton via le jeton de rafraîchissement, sans re-saisie des identifiants. Le stockage en cookie `httpOnly` protège en outre les jetons d'un accès par JavaScript (mitigation XSS). Le contrôle d'accès (RBAC) est appliqué côté serveur par un middleware d'authentification, et reflété côté client en masquant les actions non autorisées (par exemple, le bouton de modération n'est visible que pour les modérateurs et administrateurs).

7.2 PUBLICATION ET FIL D'ACTUALITÉ

La création de post accepte un texte (280 caractères), des tags, et un média optionnel **image ou vidéo** : le fichier est d'abord envoyé au service Media, puis l'URL retournée est attachée au post. Le fil chronologique agrège les posts des comptes suivis ; un *mapper* dédié transforme le document MongoDB en modèle d'affichage et résout l'auteur (identifiant → nom).

7.3 INTERACTIONS SOCIALES

Les likes utilisent une **mise à jour optimiste** : l'interface bascule immédiatement puis se recalcule sur la réponse du serveur (ou annule en cas d'erreur). Commentaires et réponses imbriquées sont stockés directement dans le document du post. Le suivi d'un utilisateur met à jour la table relationnelle des abonnements et déclenche une notification. Une section **suggestions** propose des comptes à suivre (en excluant soi-même, les comptes déjà suivis et les comptes bloqués).

7.4 BLOCAGE ET CONFIDENTIALITÉ

Un utilisateur peut en **bloquer** un autre. Le blocage est **mutuel** dans ses effets : les deux abonnements éventuels sont rompus, l'envoi *et* la consultation de messages privés deviennent impossibles entre les deux comptes (réponse 403 côté serveur, zone de saisie désactivée côté client), et les **publications du compte bloqué ne sont plus visibles** (ni dans le fil, ni sur son profil). Le contrôle est appliqué côté serveur (sur les services posts, messages et utilisateurs) et reflété immédiatement côté interface.

7.5 NOTIFICATIONS ET MESSAGERIE

Les notifications (mentions, likes, commentaires, nouveaux abonnés, messages privés) sont remontées par un hook partagé entre l'en-tête et la barre latérale, qui résout les expéditeurs et formate les libellés. Faute de WebSocket, la fraîcheur est assurée par une **interrogation périodique** (*polling*) qui actualise notifications et compteurs sans rechargement.

La messagerie privée regroupe les échanges par conversation. Elle affiche un **indicateur de présence** (pastille verte « en ligne » / grise « hors ligne », calculée à partir d'un `last_active` rafraîchi régulièrement), des **accusés de lecture** sur ses propres messages (simple coche « envoyé » / double coche « lu »), et une **pastille de messages non lus** sur la navigation.

7.6 MODÉRATION

Tout utilisateur peut **signaler** un contenu ; les signalements alimentent une file consultable par les modérateurs, qui peuvent les résoudre et, le cas échéant, **suspendre ou bannir** un compte. Le bannissement est appliqué **dès l'authentification** : un compte banni se voit refuser la connexion (réponse 403), il ne peut donc plus accéder à l'application. L'espace de modération propose en outre un **filtre des utilisateurs signalés**, une recherche, et la création de comptes (réservée à l'administrateur, qui ne crée que des comptes *standards*). L'accès à cet espace est protégé côté serveur et masqué côté client aux rôles non habilités.

7.7 COUCHE D'INTÉGRATION FRONT/BACK

La liaison entre le front-end et le back-end constitue un pilier du projet. Elle repose sur :

- une **couche de services Axios** centralisant les appels, la base URL de la passerelle et l'envoi automatique des cookies d'authentification ;
- un **renouvellement silencieux** du jeton : sur réponse 401, le client rejoue la requête après avoir obtenu un nouveau jeton d'accès via le jeton de rafraîchissement, sans déconnecter l'utilisateur ;
- une **enveloppe de réponse standardisée** et des *mappers* purs convertissant les modèles back-end en modèles d'affichage ;
- la **passerelle Nginx** réglée pour le CORS, la limitation de débit adaptée à une SPA et la diffusion des médias ;
- un **déploiement Docker Compose** complet, avec un **service de données de démonstration** exécuté automatiquement au démarrage (idempotent : comptes liés, posts avec tags, médias, notifications et messages) et des outils web d'inspection des bases (Adminer, mongo-express).

8 AXES D'AMÉLIORATION ET ÉVOLUTIONS

- **Temps réel** — diffusion des notifications et des messages via WebSocket plutôt que par interrogation périodique.
- **Recherche avancée** — moteur full-text (posts, utilisateurs, tags) et fil de tendances calculé dynamiquement.
- **Observabilité** — journalisation centralisée, métriques et traces distribuées sur l'ensemble des microservices.
- **Mise à l'échelle** — orchestration Kubernetes, réplication des bases et mise en cache (Redis) des flux les plus consultés.
- **Sécurité** — HTTPS de bout en bout (terminaison TLS) et audit automatisé des dépendances.
- **Qualité** — couverture de tests end-to-end et tests de charge pour valider la promesse de scalabilité.

CONCLUSION

Breezy atteint l'objectif fixé : proposer un réseau social complet, léger et réactif, bâti sur une architecture microservices conteneurisée, sécurisée et déployable en une seule commande. Les 23 fonctionnalités prévues ont été réalisées — de l'authentification et la gestion des rôles à la modération, en passant par la publication de contenus (texte, images, vidéos), les interactions sociales, la messagerie privée et les notifications.

Sur le plan technique, le projet valide plusieurs choix structurants : une persistance polyglotte adaptée à chaque type de donnée (PostgreSQL relationnel, MongoDB documentaire) avec un volume dédié au stockage des fichiers médias, une passerelle Nginx centralisant sécurité et routage, et une authentification par jetons JWT à durée de vie courte renouvelés silencieusement. La conteneurisation Docker garantit un environnement reproductible, du poste de développement au serveur de production.

Au-delà du livrable, ce projet a permis à l'équipe de monter en compétence sur la conception d'applications distribuées, les pratiques DevOps et la conduite de projet collaborative (organisation Git, intégration continue, répartition et suivi des tâches). Les axes d'évolution identifiés — communication temps réel par WebSocket, observabilité et mise à l'échelle horizontale — tracent une feuille de route claire pour faire évoluer ce prototype abouti vers un service de production.

SITOGRAFIE

- Documentation Next.js — <https://nextjs.org/docs>
- Documentation Express.js — <https://expressjs.com>
- Documentation Docker — <https://docs.docker.com>
- Documentation PostgreSQL / Sequelize — <https://sequelize.org>
- Documentation MongoDB / Mongoose — <https://mongoosejs.com>
- Spécification Conventional Commits — <https://www.conventionalcommits.org>